

OpsBrain

Agentic DevOps Intelligence Platform

Full Team Walkthrough — Engineer A & Engineer B

September 1 – 30, 2026

What is OpsBrain?

An AI system that monitors your infrastructure, searches your documentation, and answers plain-English questions like *"Why is the website slow?"* — automatically, 24/7, without engineers checking 10 tools manually.

Full tech stack

Linux	Git & GitHub	Docker	Kubernetes
CI/CD	AWS	Terraform	Monitoring

1. Project Overview

OpsBrain is a multi-agent AI platform built on top of the full DevOps toolchain. Instead of engineers manually checking Grafana, AWS, GitHub, logs and documentation separately, OpsBrain's agents work together autonomously to answer infrastructure questions in plain English.

The problem it solves

Manual toil: Engineers check 5–10 tools every time something breaks — Grafana, AWS Console, GitHub, log files, runbooks.

Slow incident response: Finding the root cause of an outage can take hours. At 3am this is a serious problem.

Knowledge silos: Runbooks, past incidents, and documentation live in different places. Nobody can find them fast enough.

Small team disadvantage: Small companies can't afford a large DevOps team. OpsBrain gives them 24/7 AI-powered coverage.

One-line summary

OpsBrain watches your infrastructure, reads your docs, and tells you exactly what broke and how to fix it — so engineers don't check 10 tools manually.

2. System Architecture

OpsBrain is made of five specialised agents that work together. A main Orchestrator receives every user question, decides which agents to call, and combines their answers into one clear response.

Orchestrator Agent	Main brain. Routes questions to other agents using LangGraph state machine. Manages conversation memory. Aggregates all agent responses into a final answer.
RAG Agent	Reads your runbooks, docs, and past incident reports. Stores them in a vector database. When asked a question, finds the most relevant documents and uses GPT-4o to answer.
Monitoring Agent	Connects to Prometheus and Grafana. Can read CPU, memory, traffic metrics and active alerts. Answers questions like 'Which pod is consuming the most memory?'
Infra Agent	Checks the state of AWS resources and Kubernetes pods. Can read Terraform plans. Answers questions about infrastructure health and configuration.
Code Agent	Reads GitHub pull requests, checks CI/CD pipeline status, and can trigger GitHub Actions workflows. Answers questions like 'Did the last deployment succeed?'

Message flow

User asks question → Orchestrator reads it → Router picks which agents to call → Agents run in parallel via Kafka message bus → Results come back → Orchestrator combines them with GPT-4o → Final answer returned to user.

3. Full Tech Stack

AI / Agent Layer

LangGraph	Orchestrator state machine and agent workflow routing
LangChain	Agent tool calling, RAG pipeline, prompt management
OpenAI GPT-4o	Main LLM — answers questions, reasons over retrieved data
pgvector	Vector database for storing and searching embedded documents
OpenAI Embeddings	Converts documents and queries into vectors for similarity search

Backend

Python 3.11	Primary programming language for all agents
FastAPI	REST API framework — each agent is a separate FastAPI microservice
Pydantic	Data validation and typed models for agent messages
Uvicorn	ASGI server that runs FastAPI in production

DevOps Stack

Linux	Base OS for all containers and EC2 nodes on AWS
Git & GitHub	Version control — monorepo for all agents, Terraform and K8s configs
Docker	Containerises every agent as its own isolated service
Docker Compose	Runs the entire system locally with a single command
Kubernetes (EKS)	Production deployment on AWS — handles autoscaling and self-healing
Helm	Packages Kubernetes configs for clean, repeatable deployments
GitHub Actions	CI/CD — builds, tests and pushes Docker images on every git push
ArgoCD	GitOps delivery — auto-deploys to Kubernetes when code changes
AWS	EKS, ECR, S3, RDS, DynamoDB, Secrets Manager, CloudWatch
Terraform	Provisions ALL AWS infrastructure as code — nothing clicked manually
Prometheus	Scrapes metrics from all agents and infrastructure
Grafana	Dashboard for visualising metrics from Prometheus
Loki	Log aggregation — collects logs from all agent pods

Message Queue

Apache Kafka	Async message bus — agents publish tasks and consume results
---------------------	--

Storage

PostgreSQL + pgvector	Agent memory and RAG vector store
S3 + DynamoDB	Document storage and application state
HashiCorp Vault	Secrets management — API keys, database passwords

4. Project Folder Structure

Every file has a clear purpose. Here is what lives where:

```
opsbrain/
├── agents/
│   ├── rag/
│   │   ├── main.py          ← FastAPI app entry point
│   │   ├── agent.py         ← LangGraph RAG agent logic
│   │   ├── retriever.py     ← Vector search against pgvector
│   │   ├── ingestor.py      ← Reads docs, chunks, embeds, stores
│   │   └── Dockerfile
│   ├── monitoring/
│   │   ├── main.py          ← FastAPI app
│   │   ├── agent.py         ← Prometheus + Grafana query tools
│   │   └── Dockerfile
│   ├── infra/
│   │   ├── main.py
│   │   ├── agent.py         ← AWS + Kubernetes state reader
│   │   └── Dockerfile
│   └── code/
│       ├── main.py
│       ├── agent.py         ← GitHub + CI/CD tools
│       └── Dockerfile
├── orchestrator/
│   ├── main.py              ← FastAPI entry point
│   ├── graph.py             ← LangGraph state machine
│   ├── router.py            ← Picks which agents to call
│   └── Dockerfile
├── shared/
│   ├── models.py            ← Shared Pydantic data models
│   ├── kafka_client.py      ← Kafka producer/consumer
│   └── config.py            ← Shared config and env vars
├── infra/terraform/
│   ├── main.tf              ← AWS provider
│   ├── eks.tf               ← EKS cluster
│   ├── rds.tf               ← PostgreSQL on RDS
│   ├── s3.tf                ← S3 buckets
│   ├── vpc.tf               ← VPC and networking
│   └── variables.tf
├── k8s/                     ← Kubernetes manifests for each agent
├── .github/workflows/
│   ├── ci-cd.yml            ← GitHub Actions pipeline
├── docs/
│   ├── docker-compose.yml   ← Runbooks ingested by RAG agent
│   ├── .env.example
│   └── README.md
```

5. Engineer A - Responsibilities

Engineer A shares the full platform and DevOps workload with Engineer B. Engineer A specifically owns the **RAG Agent** and **Monitoring Agent**. The **Orchestrator Agent** is built jointly by **Engineer A and Engineer B**. Shared work includes repository setup, Docker, Kafka, CI/CD, AWS/Terraform, Kubernetes/Helm, ArgoCD, observability, integration, testing, and documentation.

Week 1 - Sept 1-7

Day 1 (shared)	Set up the GitHub repo, folder structure, .gitignore, .env.example, README, and initial project skeleton together with Engineer B.
Day 2	Own the RAG document ingestor: read .md/.txt files from /docs, chunk them, create embeddings, and store them in pgvector. Also review the shared docker-compose setup with Engineer B.
Day 3	Own the RAG retriever + LangGraph RAG agent + FastAPI endpoint. Pair with Engineer B on Dockerfiles and local container validation.

Week 2 - Sept 8-14

Day 8	Own the Monitoring Agent: connect to Prometheus/Grafana and implement tools for metrics, alerts, CPU, memory, and pod health. Begin the Orchestrator design jointly with Engineer B.
Day 9	Test the Monitoring Agent end-to-end. Pair on the shared Orchestrator graph/router contract and connect RAG + Monitoring to it.
Day 10	Build the Orchestrator jointly with Engineer B: LangGraph state machine, routing, parallel calls, aggregation, and conversation memory. Review the shared GitHub Actions pipeline.

Week 3 - Sept 15-21

Day 15 (shared)	Implement/review shared Kafka producer-consumer utilities and message models. Both engineers own the communication backbone.
Day 16 (shared)	Connect every agent to Kafka and integrate the jointly built Orchestrator with all agent services.
Day 17 (shared)	Run the full async pipeline end-to-end and fix routing, timeout, serialization, or agent-integration issues together.

Engineer A - Shared Platform / DevOps Responsibilities

In addition to owning RAG and Monitoring, Engineer A actively contributes to every platform responsibility below with Engineer B. These are **shared responsibilities**, not isolated handoffs.

Docker / Compose	Co-maintain docker-compose.yml, service networking, Postgres + pgvector, Kafka, Prometheus, and Dockerfiles for all agents.
CI/CD	Co-maintain GitHub Actions: tests, Docker image builds, ECR pushes, and release checks.
AWS / Terraform	Co-design and review Terraform for VPC, EKS, RDS, S3, IAM, and Secrets Manager; participate in terraform plan/apply validation.
Kubernetes / Helm	Co-maintain agent Deployments, Services, HPAs, probes, resource limits, ConfigMaps/Secrets, and Helm charts.
ArgoCD	Co-configure GitOps deployment from GitHub to EKS and verify automatic sync behavior.
Observability	Own Monitoring Agent logic and co-maintain Prometheus scraping, Grafana dashboards, Loki logging, latency/error panels, and alerts.
Integration	Jointly own Orchestrator integration, Kafka contracts, API schemas, shared models, end-to-end tests, and failure handling.
Docs / Demo	Co-author README/runbooks, architecture notes, setup instructions, final demo, and final repository polish.

Week 4 - Sept 22-30

Day 22 (shared)	Set up/review Prometheus scraping for all agents and jointly validate Terraform infrastructure with Engineer B.
Day 23 (shared)	Configure/review Grafana dashboards while jointly validating Kubernetes manifests and Helm charts.
Day 24 (shared)	Configure/review Loki structured logging while jointly validating ArgoCD GitOps deployment.
Sept 25-28	Buffer for testing, fixes, documentation, code review, and cross-training across all shared platform work.
Sept 29-30 (shared)	Final end-to-end test, README polish, demo recording, architecture review, and final merge/push with Engineer B.

Key files Engineer A owns or co-writes

agents/rag/ingestor.py	Engineer A owns: reads docs, chunks them, embeds them, and stores vectors in pgvector.
agents/rag/retriever.py	Engineer A owns: similarity search against pgvector.
agents/rag/agent.py	Engineer A owns: LangGraph RAG agent and tool use.
agents/rag/main.py	Engineer A owns: FastAPI entry point for RAG.
agents/monitoring/agent.py	Engineer A owns: Prometheus/Grafana query tools.
agents/monitoring/main.py	Engineer A owns: Monitoring Agent FastAPI service.
orchestrator/graph.py	Shared A+B: LangGraph state machine and aggregation flow.
orchestrator/router.py	Shared A+B: routing logic and multi-agent selection.
orchestrator/main.py	Shared A+B: Orchestrator FastAPI entry point.
shared/kafka_client.py + shared/models.py	Shared A+B: Kafka messaging and request/response models.
docker-compose.yml + agents/*/Dockerfile	Shared A+B: local environment and agent containers.
.github/workflows/ci-cd.yml	Shared A+B: tests, builds, and ECR push pipeline.
infra/terraform/*	Shared A+B: AWS infrastructure as code.
k8s/* + Helm charts	Shared A+B: Kubernetes deployment, services, autoscaling, and packaging.
Prometheus / Grafana / Loki configs	Shared A+B; Engineer A leads monitoring-agent-specific observability logic.

6. Engineer B - Responsibilities

Engineer B shares the full platform and DevOps workload with Engineer A. Engineer B specifically owns the **Infra Agent** and **Code Agent**. The **Orchestrator Agent** is built jointly by **Engineer A and Engineer B**. Engineer B keeps the original DevOps/infra responsibilities, but they are now collaborative rather than exclusive.

Week 1 - Sept 1-7

Day 1 (shared)	Set up the GitHub repo, folder structure, .gitignore, .env.example, README, and initial project skeleton together with Engineer A.
Day 2 (shared)	Lead/co-build docker-compose.yml with PostgreSQL + pgvector, Kafka, Prometheus, and agent stubs. Engineer A reviews and contributes.
Day 3 (shared)	Lead/co-build Dockerfiles for all agents and validate clean builds and FastAPI startup with Engineer A.

Week 2 - Sept 8-14

Day 8 (shared)	Begin the Orchestrator jointly with Engineer A: define LangGraph state, router contract, agent response schema, and aggregation flow.
Day 9 (shared)	Pair on Orchestrator integration for RAG + Monitoring and verify end-to-end routing while Engineer A finishes Monitoring tests.
Day 10 (shared)	Co-build Orchestrator parallel routing and conversation memory. Lead/co-maintain GitHub Actions CI/CD with Engineer A.

Week 3 - Sept 15-21

Day 15	Own the Infra Agent: Kubernetes state via Python client plus AWS boto3 tools for EC2/RDS and infrastructure health.
Day 16	Own the Code Agent: GitHub REST API tools for PRs, commits, workflow runs, deployment status, and manual workflow triggering.
Day 17 (shared)	Connect Infra + Code to Kafka and the jointly built Orchestrator. Run the full multi-agent flow with Engineer A.

Engineer B - Shared Platform / DevOps Responsibilities

Engineer B retains the original DevOps and infrastructure work, while Engineer A now participates in the same responsibilities. Engineer B continues to lead many of these implementation tasks, but ownership is shared at the team level.

Docker / Compose	Co-maintain docker-compose.yml and Dockerfiles; verify all services start locally and communicate correctly.
CI/CD	Lead/co-maintain GitHub Actions with Engineer A: tests, builds, ECR pushes, and release checks.
AWS / Terraform	Lead/co-maintain Terraform for VPC, EKS, RDS, S3, IAM, and Secrets Manager; review plans and apply safely with Engineer A.
Kubernetes / Helm	Lead/co-maintain manifests, HPAs, probes, resource limits, services, and Helm charts with Engineer A.
ArgoCD	Lead/co-configure GitOps deployment and automatic sync from GitHub to EKS.
Observability	Co-maintain Prometheus, Grafana, Loki, dashboards, and alerts; Engineer A owns Monitoring Agent query logic.
Integration	Jointly own Kafka contracts, shared models, Orchestrator integration, end-to-end testing, and platform reliability.
Docs / Demo	Co-author docs, runbooks, architecture notes, setup instructions, final demo, and release polish.

Week 4 - Sept 22-30

Day 22 (shared)	Lead/co-build Terraform infrastructure while Engineer A participates in review, validation, and Prometheus scraping work.
Day 23 (shared)	Lead/co-build Kubernetes manifests + Helm while Engineer A participates and co-validates Grafana dashboards.
Day 24 (shared)	Lead/co-configure ArgoCD while Engineer A participates and co-validates Loki log collection.
Sept 25-28	Buffer for testing, fixes, documentation, code review, and cross-training across shared responsibilities.
Sept 29-30 (shared)	Final end-to-end test, README polish, demo recording, architecture review, and final merge/push with Engineer A.

Key files Engineer B owns or co-writes

agents/infra/agent.py	Engineer B owns: AWS boto3 + Kubernetes client tools for the Infra Agent.
agents/infra/main.py	Engineer B owns: Infra Agent FastAPI service.
agents/code/agent.py	Engineer B owns: GitHub API and CI/CD inspection/trigger tools.
agents/code/main.py	Engineer B owns: Code Agent FastAPI service.
orchestrator/graph.py	Shared A+B: LangGraph state machine and aggregation flow.
orchestrator/router.py	Shared A+B: routing logic and multi-agent selection.
orchestrator/main.py	Shared A+B: Orchestrator FastAPI entry point.
docker-compose.yml	Shared A+B; Engineer B may lead initial implementation.
agents/*/Dockerfile	Shared A+B; Engineer B may lead initial implementation.
.github/workflows/ci-cd.yml	Shared A+B; Engineer B leads/co-maintains pipeline implementation.
infra/terraform/eks.tf + vpc.tf + rds.tf + s3.tf	Shared A+B; Engineer B leads/co-maintains infrastructure implementation.
k8s/* + Helm charts	Shared A+B; Engineer B leads/co-maintains deployment definitions.
ArgoCD config	Shared A+B; Engineer B leads/co-maintains GitOps setup.
Prometheus / Grafana / Loki configs	Shared A+B; Engineer A leads monitoring-agent-specific observability logic.
shared/kafka_client.py + shared/models.py	Shared A+B: messaging contracts and common models.

7. Full September Calendar - Revised Ownership

Both engineers participate in all core platform responsibilities. Agent ownership is split: **Engineer A = RAG + Monitoring**; **Engineer B = Infra + Code**; **Orchestrator = shared A+B**.

Sept 1	Project setup + GitHub repo (shared)	Project setup + GitHub repo (shared)
Sept 2	RAG ingestor + review docker-compose	Lead/co-build docker-compose + review RAG environment
Sept 3	RAG retriever/agent + co-test Dockerfiles	Lead/co-build Dockerfiles + co-test RAG container
Sept 4-7	Buffer / testing / code review (shared)	Buffer / testing / code review (shared)
Sept 8	Monitoring Agent + Orchestrator design (shared)	Orchestrator design (shared) + platform support
Sept 9	Test Monitoring + integrate RAG/Monitoring	Co-build Orchestrator + integrate RAG/Monitoring
Sept 10	Orchestrator routing/memory + CI/CD review	Orchestrator routing/memory + lead/co-build CI/CD
Sept 11-14	Buffer / testing / code review (shared)	Buffer / testing / code review (shared)
Sept 15	Kafka shared + platform integration	Infra Agent + Kafka shared
Sept 16	Connect agents to Kafka + integration	Code Agent + connect agents to Kafka
Sept 17	Full async pipeline test (shared)	Full async pipeline test (shared)
Sept 18-21	Buffer / testing / code review (shared)	Buffer / testing / code review (shared)
Sept 22	Prometheus scraping + Terraform review	Lead/co-build Terraform + Prometheus review
Sept 23	Grafana + K8s/Helm review	Lead/co-build K8s/Helm + Grafana review
Sept 24	Loki + ArgoCD review	Lead/co-build ArgoCD + Loki review
Sept 25-28	Buffer / polish / documentation (shared)	Buffer / polish / documentation (shared)
Sept 29-30	Final demo + README + push (shared)	Final demo + README + push (shared)

8. Project Management - Linear

Use Linear (linear.app) instead of Jira. Create one issue for every task in the revised calendar above.

Assign ownership	Agent-specific issues: Engineer A owns RAG + Monitoring; Engineer B owns Infra + Code. Orchestrator issues are assigned to both / paired work.
Shared platform issues	Docker, Kafka, CI/CD, Terraform, Kubernetes/Helm, ArgoCD, Prometheus/Grafana/Loki, integration, testing, and documentation are shared responsibilities. Use a lead + reviewer/subtask pattern so both engineers contribute.
Cycles	Use one weekly cycle. Keep shared tasks visible in both engineers' workload rather than treating them as one-person handoffs.
Code review	For owned-agent work, the other engineer reviews the PR. For shared platform/orchestrator work, use paired branches or small PRs with alternating lead/reviewer roles.

Git workflow for the team

main branch	Always working and deployable. Never push directly to main.
Engineer A agent branches	Examples: feature/rag-agent, feature/monitoring-agent.
Engineer B agent branches	Examples: feature/infra-agent, feature/code-agent.
Shared Orchestrator branches	Examples: feature/orchestrator-router, feature/orchestrator-memory. Both engineers pair and review.
Shared platform branches	Examples: feature/docker-setup, feature/kafka, feature/terraform-eks, feature/k8s-helm, feature/observability.
Pull requests	The non-leading engineer reviews before merge. Rotate lead/reviewer on shared responsibilities.
Daily sync	10-minute sync: finished work, today's work, blockers, and shared integration dependencies.

9. Day 1 Checklist - Sept 1 (Revised)

Both engineers do the complete project setup together. Neither Engineer A nor Engineer B is limited to only AI or only DevOps setup.

Both	Create GitHub repo named opsbrain - public, no default files.
Both	Clone the repo locally and create the full project folder structure.
Both	Create .gitignore, .env.example, README.md, docker-compose.yml placeholder, and docs/sample-runbook.md.
Both	Set up Python virtual environments and install the shared Python dependencies.
Both	Verify Docker and Docker Compose are installed and running.
Both	Bring up the docker-compose stub and confirm the base services start without errors.
Both	Create local .env files and verify required API/config variables. Engineer A will lead later RAG/Monitoring API use; Engineer B will lead later Infra/Code integrations.
Both	Agree on shared Pydantic message schemas, Kafka topic naming, API conventions, branch naming, and PR review rules.
Both	Sketch the Orchestrator contract together: input state, router output, agent response schema, aggregation result, and error path.
Both	Push the initial commit and open the first shared setup PR/review.

Ownership reminder: Engineer A owns RAG + Monitoring Agents. Engineer B owns Infra + Code Agents. Orchestrator and all core platform/DevOps responsibilities are shared.

10. Definition of Done — Sept 30

By the end of September 30, OpsBrain should pass every item below:

- You can ask 'Why is the website slow?' and get an intelligent answer from multiple agents working together
- The RAG agent can search your runbooks and return relevant answers
- The Monitoring Agent can query Prometheus and tell you which pod has high CPU
- The Infra Agent can describe the state of your Kubernetes cluster
- The Code Agent can tell you whether the last GitHub Actions run succeeded
- The Orchestrator correctly routes questions to the right agents and combines their answers
- All agents communicate via Kafka asynchronously
- docker-compose up starts the whole system locally with one command
- GitHub Actions automatically builds and pushes Docker images on every push to main
- Terraform provisions the full AWS infrastructure: EKS, VPC, RDS, S3
- ArgoCD automatically deploys to Kubernetes when code is pushed to main
- Prometheus is scraping metrics from all agents
- Grafana dashboards show query latency, error rates, and agent health
- Loki is collecting logs from all agent pods
- The GitHub repo has a clean README explaining the project

Congratulations. If all boxes above are checked, you have built a production-grade Agentic AI + DevOps platform using the full modern stack: Linux, Git, Docker, Kubernetes, CI/CD, AWS, Terraform, and Monitoring — with AI agents on top.